

Data-Driven Dynamics, Final Project

Implementing Time-Input State Aware Bayesian Calibration (TISAC) Methods into Real-Time Control to Solve the Comma AI Controls Challenge

Alexander Knecht

Department of Mechanical Engineering

Clemson University

Clemson, SC

knecht@clemson.edu

Abstract—Modern data-driven dynamic control of non-linear systems is a complex problem, with many well studied approaches, each with their own strengths and weaknesses. In this project, we approach the Comma AI controls problem through the lens of Bayesian statisticians, as that is where my primary area of research currently lies. A unique formulation to solve the data-driven control problem is developed, leveraging 2 distinct features from my current field of study. The first is the unique Fourier lifter that is chosen as the basis function. This is a method that has shown significant promise in early testing of data-driven *calibration* techniques, and therefore I wanted to see if it could be successful in *controls* applications. The second uniquely Bayesian-inspired element of my formulation is the introduction of uncertainty quantification techniques into the control approach. By generating an ensemble of 50 potential Koopman operators at each control step and including the variance produced by each ensemble directly into the cost function, we are able to minimize predictive uncertainty at each discrete point along the control path, capitalizing on predictive confidence. Although this method will require a bit more development to beat the baseline PID due to the complexity of the design and the performance dependence on hyperparameters, it is already able to achieve a benchmark total cost of 92, clearing the leaderboard cost threshold of 110, and beating the baseline PID controller in the primary tracking metric of lateral acceleration cost.

Index Terms—Koopman Operator, MPC, Data-Driven Dynamics

I. INTRODUCTION

Modern autonomous vehicle controllers rely heavily on predictive models to calculate optimal steering actions. However, static, data-driven models can suffer from performance degradation under sudden condition shifts (neighborhood driving to highways, varying road, weather, lighting conditions, etc). The concept of TISAC (time-input, state-aware, Bayesian-inspired calibration)

is a novel method of capturing spatiotemporal dynamics that has only been applied to offline, static calibration campaigns. The fundamental concepts that inspired TISAC are incredibly relevant to controls problems. It would be unreasonable to assume that relevant system information, determined through an initial calibration campaign, would hold true for all time, therefore TISAC allows for time-dependent evolution of system behavior. Factors such as tire pressure, tread-wear, and vehicle alignment will drift over extended periods of operation, therefore necessitating the introduction of time-dependent states. While these specific factors will likely not directly have any influence in the Comma AI controls challenge due to the brevity of the test segments, it provides an environment in which the concept may be tested to give initial confidence that it is worth exploring further.

The goal of this project, therefore, is to produce an online and real-time control system utilizing the TISAC framework, treating the Koopman matrix as the calibration parameter tuned through a time-input Bayesian inference campaign. By setting a probability distribution of 50 potential Koopman operators at each point in time, the system is able to adapt the control logic in real time to varying conditions. In this project, this framework is embedded into a controller and evaluated using the Comma AI challenge, which provides a full evaluation toolkit to quantify control system performance against a baseline controller (PID).

NOTE: The primary concept behind TISAC is part of the work performed by Dr Sez Russcher’s graduate students. The first TISAC publication (for offline, static data) has been submitted to a publisher. We ask the work discussed in this project not be disseminated until the paper has been published.

II. A BRIEF REVIEW OF RELEVANT LITERATURE

There have been several recent studies on time-input calibration of computer models. Frameworks such as the Time-Input Bayesian Regression (TIBR) (Higdon et al, 2004) directly treat time as an input value, monitoring and learning how outputs can change across time (Juang et al, 2013).

Unlike the traditional TIBR, TISAC treats time as a covariate existing alongside spatial location x . In addition, temporal smoothness is applied via the priors (Durbin & Koopman 2012) over t . Taking this time-as-covariate perspective into consideration, Cho et al. (2024) leveraged Bayesian optimization to consider temporal effects by embedding spatiotemporal kernels. Alternatively, run-indexed approaches have been proposed to cover run-to-run process drifts, using non-stationary Wiener process kernels to accommodate run indices, instead of time. This effectively allows the model to learn patterns across runs.

There are two generally accepted methods of capturing time-dependent behavior in Gaussian Processes (GPs):

- (i) Sequential Learning: The model gets updated as each new data point arrives, thereby adapting swiftly to drift with minute parameter changes. Such an arrangement is latency-friendly; however, it can lead to a higher bias accumulation and needs extra engineering to encode physics. (West & Harrison, 1997; Doucet, de Freitas, & Gordon, 2001)
- (ii) Bayesian Model Averaging (BMA): Considers an ensemble approach, by combining multiple models using the posterior weights to improve calibration. However, this increases the computation cost significantly (Hoeting, Madigan, Raftery, & Volinsky, 1999; Raftery, Gneiting, Balabdaoui, & Polakowski, 2005).

BMA, although robust, can be computationally heavy. TISAC shares information across the spatiotemporal domain with Fourier features whereas BMA, due to its ensemble nature, spreads the information sharing capacity across models, demanding more data or carefully adjusted priors to reach the same fidelity. In contrast, TISAC exhibits consistent performance even when spatiotemporal data is scarce.

The originally proposed TISAC architecture is developed for offline batch calibration of time-series data. For instance, it has been successfully applied to battery degradation (time-varying states, but consistently observ-

able status with no dynamic inputs) models with a very high degree of precision. This extension seeks to apply it to real-time, online control, as outlined in the following methodology. Of primary interest is the move away from GPs to map to the spatiotemporal domain. Based on the need for real-time control, and the known latency of GPs, the proposed architecture replaces the GP with a Fourier-Lifted Koopman Operator via an Ensemble Kalman Filter. This provides tremendous speed up, linearization, and extends the domain of applicability to real-time control.

III. COMMA AI CONTROLS CHALLENGE

A. Background

In order to understand the proposed solution method, it is critical to understand the background of the Comma AI controls challenge, as well as the scoring methods used. Even in the early development of this project, significant time was wasted chasing a lower Jerk Cost than Lateral Acceleration cost, due to an initial misunderstanding of system scoring.

This challenge is an open-source machine learning and control theory test that allows students, researchers, and hobbyists to test their own controller designs on an autonomous vehicle's lateral dynamics. Rather than relying on a kinematic or physics model, this challenge is uniquely difficult due to its 'black box' dynamics, housed in a neural simulator called TinyPhysics. This simulator is trained on vast amounts of real-world, noisy driving data gathered from numerous vehicles. By learning directly from data, the model captures the complexity of vehicle dynamics, including the influences of characteristics such as road camber, actuator latency, and environmental noise adding a significant layer of challenge to those developing effective solutions.

Using the available 20,000 data sets, users have access to the following information at each time step:

- Ego Velocity
- Longitudinal Acceleration
- Lateral Acceleration (introduced by road camber)
- Vehicle Lateral Acceleration
- Controller Steering Action

The objective then is for challenge participants to design a controller that outputs an optimal steering command at each time step to create the desired lateral trajectory, while minimizing costs associated with jerk and lateral acceleration.

B. Scoring Technique

The performance of each submitted controller is graded using a cost function built by the Comma AI developers that looks to balance trajectory tracking accuracy with passenger comfort. The evaluation is conducted over a defined number of discrete simulation steps (N), strictly penalizing large trajectory deviations (lateral acceleration cost) and high jerk (rapid, jerky changes in acceleration). The lateral acceleration is computed by taking the scaled Mean Squared Error (MSE) between the vehicle's *true* lateral acceleration $a_{lat,true}$ and the *target* lateral acceleration, $a_{lat,target}$. The lateral acceleration cost is then given as:

$$C_{lat} = \frac{\sum_{t=1}^N (a_{lat,true} - a_{lat,target})^2}{N} \times 100$$

To ensure ride smoothness and penalize large or erratic steering corrections, the jerk cost is calculated based on the rate of change of the lateral acceleration over the simulation time step, Δt :

$$C_{jerk} = \frac{\sum_{t=2}^N \left(\frac{a_{lat,t} - a_{lat,t-1}}{\Delta t} \right)^2}{N - 1} \times 100$$

Finally, the total cost is computed as a weighted sum of these two metrics, penalizing any deviation from adherence to the true path much more severely than rider comfort, with a formulation of:

$$C_{Total} = 50C_{lat} + C_{jerk}$$

The ultimate goal of participating in this challenge is to develop a controller that minimizes C_{total} as much as possible, with the most successful controllers scoring well under the target value of 100.

IV. TISAC BACKGROUND

Injecting Time-Input State-Aware Bayesian-Inspired Calibration into Real Time, Data-Driven Vehicle Dynamics has been a very challenging, multi-step process. A brief synopsis of the three primary steps that make this approach unique are outlined below.

- Step 1: Offline prior. The Koopman matrix K is trained offline on historic datasets provided from Comma AI open-loop driving data. K is a compact linear summary of the vehicle's physics, with K_x capturing the lateral acceleration decay, K_u describing how much the associated steering action moves the lateral acceleration per step, and K_d capturing

how speed and road roll contribute to the lateral acceleration. This becomes what is referred to in calibration campaigns as the *prior*, or the initial belief about the system. The K matrix parameters, denoted by $\theta(x,t)$ are then calibrated using a Bayesian framework as outlined in step 2.

- Step 2: Bayesian adaptation (EnKF). At runtime, instead of a single K , 50 unique K matrices are generated in a normally-distributed probability cloud. Every 100ms the true lataccel is observed and compared to what each of the 50 matrices predicted, shifting the probability cloud toward those closest to the truth. This is where TISAC and the Bayesian Framework supply a very unique approach, the calibration parameters (K) are tracked as a slowly-evolving stochastic field, updated recursively as new data arrives.
- Stage 3: Risk-aware control (SMPC). Rather than picking the single best K and planning against it, all 50 potential futures are presented simultaneously to the controller. The optimizer picks the steer command that minimizes tracking error and the variance across *all* 50 futures, embedding caution when the ensemble disagrees with reality and aggression when it is confident through a custom-built cost function.

A flow chart, built to aid in visualization of this complex, multi-step process is included after the methodology section (Figure 1).

V. METHODOLOGY

A. Fourier-Lifted State Space

Classical data-driven controllers may fail to capture highly non-linear vehicle dynamics efficiently enough to run at very high frequencies (in this case, 10Hz). To bypass this computational burden, it has become standard to lift higher-dimensional spaces into a space where they may be approximated linearly.

For this project, we allow the physical state of the vehicle to be the lateral acceleration, $y_k \in \mathbb{R}$, at time step k . The state is then expanded using a Fourier basis (per the TISAC architecture) to capture both broad curves *and* high-frequency inertial vibrations. The lifting function, $\Phi : \mathbb{R} \rightarrow \mathbb{R}^7$ is defined as:

$$x_k = \Phi(y_k) = \begin{bmatrix} y_k \\ \sin(\pi y_k) \\ \cos(\pi y_k) \\ \sin(2\pi y_k) \\ \cos(2\pi y_k) \\ \sin(4\pi y_k) \\ \cos(4\pi y_k) \end{bmatrix}$$

The inputs that direct system response are the steering command, $u_k \in \mathbb{R}$, and the measured physical disturbances, $d_k \in \mathbb{R}^3$, representing longitudinal velocity (v_{ego}), longitudinal acceleration (a_{ego}), and the bank angle of the road, measured in radians ($roll$). To prevent large-magnitude variables from dominating the covariance calculations, v_{ego} is normalized against an arbitrarily chosen speed, originally picked at 30 m/s, but later reduced to 15 m/s following a brief optimization study.

The complete system vector $\Omega_k \in \mathbb{R}^{11}$ may now be constructed as:

$$\Omega_k = \begin{bmatrix} x_k \\ u_k \\ d_k \end{bmatrix}$$

B. Offline Koopman Training

Before initializing online control, a linear representation of the vehicle dynamics, serving as the *prior* for the Bayesian formulation, must be established. To build this baseline model, data is loaded from a single reference segment. The lifting function is then applied to the time-series snapshots to produce the sequential lifted state matrices, $X_{1,\text{lifted}} \in \mathbb{R}^{7 \times N}$ and $X_{2,\text{lifted}} \in \mathbb{R}^{7 \times N}$.

To accurately capture the coupled effects of the environment and the control authority, an augmented matrix Ω is constructed by vertically stacking the lifted state, steering input, and disturbance vector for each corresponding time step. The Koopman operator K is then computed via the closed-form Tikhonov-regularized least squares solution to ensure stability and prevent overfitting:

$$K = X_{2,\text{lifted}} \Omega^T (\Omega \Omega^T + \lambda I)^{-1}$$

Because the original state variable is mapped directly to the first element of the lifted state vector, the observation matrix $C = e_1^T$ is exact by construction. This is an important element of this lifting design, as it theoretically guarantees no reconstruction error for the primary tracking variable, without relying on an approximated output mapping.

Finally, the resulting Koopman operator $K \in \mathbb{R}^{7 \times 11}$ is split into mathematically distinct block matrices governing the linear transition:

$$x_{k+1} = K_x x_k + K_u u_k + K_d d_k$$

C. Real-Time Bayesian Adaptation via Ensemble Kalman Filter

The novel contribution of this work is applying Bayesian, time-input, state-aware calibration techniques to the open loop control problem. In attempts to account for potentially changing environmental conditions, as well as discrepancies between the open loop training data and the true physical plant, K_{baseline} is *continuously* calibrated online using an Ensemble Kalman Filter.

The baseline matrix is flattened into a parameter vector $\theta \in \mathbb{R}^{77}$ which serves as the *prior mean*. A stochastic probability cloud of $N = 50$ unique parameter models (the ensemble in this case) is generated as:

$$\theta_{k-1}^{(i)} \sim \mathcal{N}(\theta_{\text{baseline}}, P_0), \quad i \in \{1, \dots, N\}$$

The state-transition parameters (K_x) often maintain magnitudes near 1.0, while the control-authority parameters (K_u) are orders of magnitude smaller. A uniform noise profile would gently perturb the inertial dynamics while strongly overpowering the delicate steering parameters. To avoid this, the process noise covariance is formulated as a parameter-proportional diagonal matrix, Q :

$$Q = \text{diag}(\sigma_{\text{proc}} \cdot (|\mu_\theta| + \epsilon))^2$$

Where μ_θ represents the prior mean of the parameter vector, σ_{proc} is the fractional mutation rate, and ϵ is a strictly positive lower-bound scalar.

At each 100ms control step, we predict the next *scalar* lateral acceleration value for all N models:

$$\hat{y}_k^{(i)} = \left(K_{k|k-1}^{(i)} \Omega_{k-1} \right)_{[0]}$$

Upon observing the *true* scalar state, y_k , the cross-covariance between the parameters and the predictions, $P_{\theta y}$ and the variance of the predictions P_{yy} are computed. The Kalman gain $\mathcal{K}$ dictates the *optimal* shift:

$$\mathcal{K} = P_{\theta y} (P_{yy} + R)^{-1}$$

Each matrix in the ensemble is then updated recursively:

$$\theta_{k|k}^{(i)} = \theta_{k|k-1}^{(i)} + \mathcal{K} \left(\hat{y}_k^{(i)} - \hat{y}_k^{(i)} \right),$$

$$\tilde{y}_k^{(i)} = y_k + \nu^{(i)}, \quad \nu^{(i)} \sim \mathcal{N}(0, R)$$

where $\nu^{(i)} \sim \mathcal{N}(0, R)$ maintains the statistical spread of the ensemble.

Note on EnKF Observability: Only row 0 of K (the 11 parameters governing the lataccel prediction directly) is strictly observable. The remaining 66 parameters (rows 1–6, governing the Fourier features) receive updates through cross-covariance with row 0, presenting a partial observability challenge for the EnKF. However, the hidden Fourier parameters are successfully updated by using the historical cross-covariance with the primary tracking variables, allowing the filter to infer shifts in the higher-dimensional space from a 1D observation.

D. Risk Aware Stochastic Model Predictive Control

With a calibrated distribution of 50 potential physics models now available, the control optimizer executes a trajectory rollout over a predictive horizon H . Originally a horizon of $H = 10$ was selected, but following optimization studies it was determined that this control method actually performed much better with a shorter horizon, so a horizon of $H = 5$ is used in the final formulation. Instead of optimizing against a single mean model, the SMPC simultaneously propagates the augmented state Ω through all 50 matrices to generate a *distribution* of future trajectories. The stochastic objective function J is formulated to minimize nominal tracking error while actively penalizing regions of the state space where the physical models disagree:

$$\min_U J = \sum_{k=1}^H \left[w_{err} \left(\mu_{y,k} - r_k \right)^2 + \dots \right. \\ \left. w_{jerk} \left(\Delta u_k \right)^2 + w_{steer} \left(u_k \right)^2 + w_{uncert} \left(\sigma_{y,k}^2 \right) \right]$$

The following conditions are applied:

- $u_k \in [-2.0, 2.0]$ (Physical steering limits)
- $x_{k+1}^{(i)} = K^{(i)} \Omega_k, \quad \forall i \in \{1, \dots, 50\}$
- $\mu_{y,k} = \frac{1}{N} \sum_{i=1}^N \hat{y}_k^{(i)}$ (Mean prediction)
- $\sigma_{y,k}^2 = \frac{1}{N-1} \sum_{i=1}^N (\hat{y}_k^{(i)} - \mu_{y,k})^2$ (Ensemble variance)

The inclusion of the control effort penalty (w_{steer}) ensures the problem remains convex, acting as a mathematical centering spring. Perhaps most important to this formulation is the introduction of the *uncertainty penalty*, (w_{uncert}), which actively forces the controller to avoid uncertain actions. Due to the relationship $\text{Var}(K_u u) = (u^2) \text{Var}(K_u)$, aggressive steering inputs will amplify the prediction variance $\sigma_{y,k}^2$ of the ensemble. The optimizer, based on the cost function, will select the most advantageous combination of *smooth* and *cautious* control actions. This is particularly useful when entering unexperienced physical regimes where the EnKF ensemble lacks consensus, making an attempt at risk-aware autonomy. The inclusion of the w_{uncert} is a novel approach that introduces Bayesian calibration and uncertainty quantification techniques into the world of data-driven controls, allowing actions to be driven by probability fields rather than forced attempts at a single best answer.

Note: It is also important to note that w_{jerk} is only *indirectly* related to the actual jerk cost as evaluated by the Comma AI scoring technique. In this setting it refers to the weight with which steering rate (Δu_k) is penalized, but the actual jerk cost will be calculated downstream by the Comma AI evaluation script.

Figure 1 presents a flow chart designed to aid in the visualization of the workflow used in this control method.

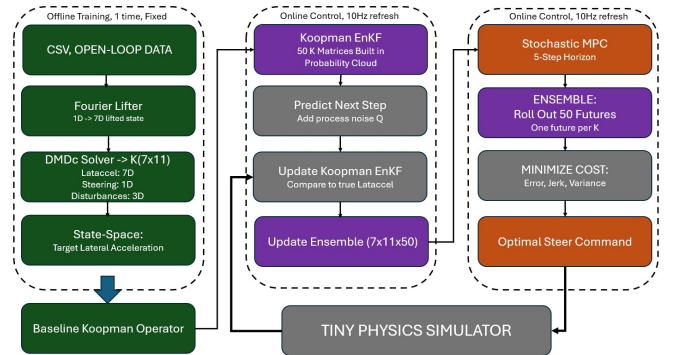


Fig. 1. In this chart, green boxes represent actions taken only once, in the leadup to the real-time control. Purple boxes are the uniquely Bayesian-inspired elements. Orange boxes represent control actions, and grey are standard internal processes.

VI. FINDINGS

The final implementation of TISAC and Bayesian methods into a data-driven controls problems was much more successful than I originally anticipated it could

be, and certainly outperformed my expectations at the semester midpoint. Prior to appropriate hyperparameter scaling, I was stuck with total costs regularly falling in the 150-160 range. The reason for this was mostly due to my own misunderstanding of the Comma AI scoring method. I had optimized my setup for the *jerk* cost, not taking the *lateral acceleration* cost to be 50x more important than jerk cost. As is strongly evidenced from Table 1, once I allowed the jerk cost to relax and optimized for the lateral acceleration metric, I was able to score significantly better, beating the leaderboard threshold of 110 and achieving close to the baseline PID. I believe with sufficient time, I would absolutely be able to tune this method to outperform the baseline PID by optimizing for both tracking and jerk, not just one or the other. I believe that perhaps if I were to collaborate with more controls-oriented students, it is possible that this method could be optimized significantly further, allowing an even more competitive score.

Controller	Lat Accel Cost	Jerk Cost	Total Cost
Leaderboard Threshold	N/A	N/A	110
Baseline (PID)	1.211	20.151	80.73
TISAC, NO Tuning	3.065	15.470	168.722
TISAC, Basic Tuning	2.900	16.375	156.15
TISAC, Optimization	1.193	32.498	92.137

Table 1 demonstrates project findings and results numerically. Demonstrated herein is the fact that the TISAC controller is highly tunable, to beat the baseline in *either* jerk cost or lateral acceleration cost, but is challenging to optimize for both. Any further development of this method would need to focus on reducing jerk cost without allowing the lateral acceleration cost to creep back up.

Figure 2 demonstrates the effectiveness of the original, offline Koopman Operator training, achieving a nearly perfect tracking accuracy. Although this is not representative of the architecture’s performance on *unseen* segments (as this segment was used for training), it does demonstrate the validity of the proposed lifting architecture.

Figure 3 shows the efficacy of this control method, shown on a single path (00100), selected at random from the catalogue of 20,000 segments. Of particular interest is the smoothness of the control curve, as well as the exceptionally close tracking to the desired target path. The single segment cost in this case is 2.046, with an exceptionally low lateral acceleration cost.

Finally, Figure 4 shows the output images from the Comma AI custom evaluation toolkit. As part of the eval-

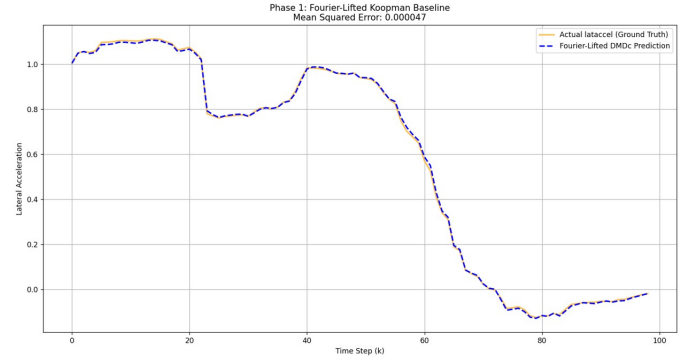


Fig. 2. Offline Training Success

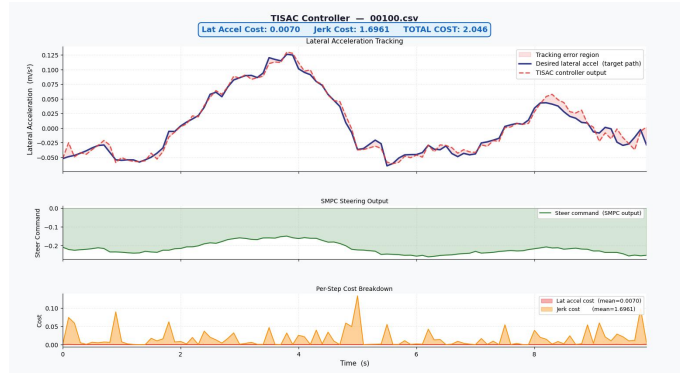


Fig. 3. Single-Run Tracking Results

uation package, Comma AI compares the performance of the custom controller (TISAC) and their baseline PID controller to determine controller success (i.e., a highly successful controller will beat the Comma AI PID). Although the TISAC score just misses that of the Comma AI PID, the tracking still performs exceptionally, as shown from Figure 4, with tracking accuracy prioritized over jerky movement elimination.

VII. CONCLUSION AND FUTURE WORK

In conclusion, TISAC and Bayesian techniques show incredible promise in their integration into online, real-time controls challenges. Comma AI provided an exceptional backdrop against which this formulation could be tested, with prepackaged data, guardrails, and evaluation toolkits. Despite early challenges, the real-time, online TISAC formulation was able to produce a viable controller, coming exceptionally close to the baseline PID score of 80.725. As the scope of this project was certainly aggressive for just a single semester, there are



Fig. 4. Lateral Acceleration using TISAC Control Architecture vs Baseline and PID Controller for test rollout visualizations on segments 0-4

many areas where this method may be further tested for efficacy and improvements, to include:

- Optimization campaigns that discover how to reduce both the lateral acceleration cost and the jerk cost simultaneously, without suffering from the trade-off that I encountered.
- Parallelization in the 50 K-Matrix roll-out procedure would provide much needed computational speedup. The current method is computationally expensive, making it less than ideal for real-time applications.
- Parameter studies to determine better guardrails on the Gaussian distribution that the K-Matrices are allowed to occupy, to determine a more optimal distribution.

VIII. A NOTE ON THE CODE SUBMISSION

The code developed for this project may be located on my public GitHub page, https://github.com/alexknecht/ME8930_Final_Project.git.

The files included are:

- train_fourier_koopman.py
- smpc.py

- enkf.py
- lifter.py
- controller.py
- plot_tisac.py
- K_baseline.npy

To use the code, download the controller.py file into the Comma AI ‘Controllers’ subfolder. All other files should be placed directly into the main Comma AI file directory. Although a baseline K matrix is already given (offline trained), this may be re-created with the command:

```
python train_fourier_koopman.py
```

After the baseline Koopman matrix has been generated, the custom Comma AI evaluation function may be executed from the main directory using the following command:

```
python eval.py \
--model_path ./models/tinyphysics.onnx \
--data_path ./data \
--num_segs 100 \
--test_controller controller \
--baseline_controller pid
```

This will generate the Comma AI report in the main directory as an HTML file, which may then be opened and evaluated. The best-achieved report is attached as a supplement to this report, after the references.

If a single run is needed, rather than the full evaluation pipeline, the simple:

```
python plot_tisac.py
```

may be used to run the controller on a single data set (selectable within the code) and to evaluate the controller’s performance on that specific segment.

Any further questions regarding usage of the code should be directed to Mr. Alexander Knecht.

IX. ACKNOWLEDGMENTS

Mr. Aryan Panchal is acknowledged for his initial contributions to the TISAC concept.

A brief note on artificial intelligence use: While AI is permitted in this class, I preferred to write all code and reports myself to ensure thorough comprehension of the topic. AI was leveraged for code debugging, cleaning up syntax, and adding comments to some of the more complex functions, but all intellectual property

contained herein is my own, written and developed without excessive reliance on external tools.

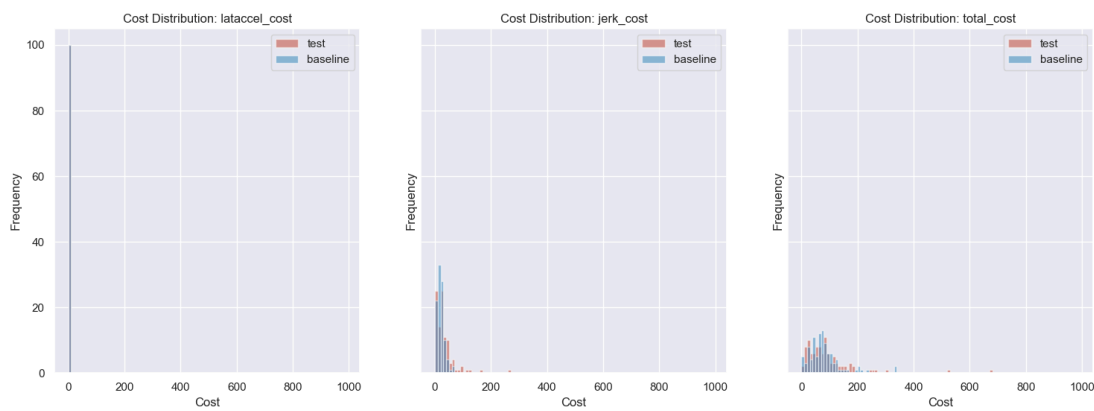
REFERENCES

- [1] Cho, K., Shao, K., & Ali, A. (2024). *Run-indexed time-varying Bayesian optimization with positional encoding for auto-tuning of controllers: Application to a plasma-assisted deposition process with run-to-run drifts*.
- [2] Doucet, A., de Freitas, N., & Gordon, N. (Eds.). (2001). *Sequential Monte Carlo Methods in Practice*. Springer New York.
- [3] Durbin, J., & Koopman, S. J. (2012). *Time Series Analysis by State Space Methods* (2nd ed.). Oxford University Press.
- [4] Higdon, D., Kennedy, M., Cavendish, J. C., Cafeo, J. A., & Ryne, R. D. (2004). Combining field data and computer simulations for calibration and prediction. *SIAM Journal on Scientific Computing*, 26(2), 448–466.
- [5] Hoeting, J. A., Madigan, D., Raftery, A. E., & Volinsky, C. T. (1999). Bayesian model averaging: A tutorial. *Statistical Science*, 14(4), 382–417.
- [6] Juang, C. H., et al. (2013). Probabilistic model calibration for soil slope under rainfall: Effects of measurement duration and frequency in field monitoring. *Engineering Geology*.
- [7] Raftery, A. E., Gneiting, T., Balabdaoui, F., & Polakowski, M. (2005). Using Bayesian model averaging to calibrate forecast ensembles. *Monthly Weather Review*, 133(5), 1155–1174.
- [8] West, M., & Harrison, J. (1997). *Bayesian Forecasting and Dynamic Models* (2nd ed.). Springer-Verlag.

Comma Controls Challenge: Report

Test Controller: controller ✂ Baseline
Controller: pid

Aggregate Costs (total rollouts: 100)



controller	lataccel_cost	jerk_cost	total_cost
baseline	1.211	20.151	80.725
test	1.193	32.498	92.137

✗ Test Controller (controller) failed to beat Baseline Controller (pid)! ✗

Sample Rollouts



Data Driven Dynamics: Final Project

**Implementing TISAC (Time-Input, State-Aware Bayesian Calibration) Methods
into Real-Time Control to Solve the Comma AI Controls Challenge**

Alexander Knecht
Select Topics in Mechanical Engineering
Spring '26, Clemson University

Overview

- Motivation
- Background
- Methodology
- Results
- Conclusion

Motivation

- TISAC has been proposed as a novel method of capturing *time-dependent* parameters in modeled systems.
- Traditional (KOH) model calibration is merely state dependent:

$$\zeta_{\theta}(x) = \eta(x, \theta) + \delta(x) + \epsilon$$

$\eta(.,.)$	simulator model
$\delta(.)$	model form error
x	application domain
θ	parameter space

- This is a problem – why?
 - Parameters may vary with time (even for “static” systems)! Extended use, varying conditions, and life limitations all impact system behavior as states evolve.
 - NO STATE IS CONSTANT! Even the best calibrated models which do not take in temporal considerations will end up biased as parameters drift.

Motivation

- TISAC addresses the issue of parameter evolution by allowing the treatment of calibration parameters as functions of **state** (x) and **time** (t):

$$Y = \eta(t, x, \theta_1(t, x), \theta_2(t))$$

- SOLUTION: TISAC explicitly models calibration parameters as temporally evolving stochastic fields constrained by physical dynamics, enabling continuous physics-consistent calibration and forecasting.
- Why use it now, for this project?
 - Initial TISAC validation tests were performed exclusively on synthetic data
 - Initial tests were performed on *offline* and *static* systems (i.e., time dependent fields but no systems in motion) such as battery life degradation, electrical grid parameter simulation, etc.

Background – Comma AI Challenge

Project Objective: Safely control nonlinear vehicle lateral dynamics by continuously calibrating a **linear surrogate model** in real-time.

Problem: Vehicle lateral dynamics (tire slip, steering compliance) are highly nonlinear and non-stationary (changing with tire wear, speed, and road bank).

Gap this Addresses: Standard PID or LQR controllers will fail to adapt to changing **SYSTEM PARAMETERS**.



Methodology

Combining TISAC architecture with data-driven dynamics learned from class to solve the Comma AI Controls Challenge:

- Allow vehicle dynamics to be governed by the discrete-time, nonlinear system:

$$x_{k+1} = F(x_k, u_k, d_k)$$

x_k : lateral acceleration

u_k : steering command

d_k : environmental disturbances (velocity, longitudinal acceleration, road bank)

- Leveraging the Koopman Operator, $\mathcal{K}$, the nonlinear system may be linearized by lifting the state using the function $\psi(x)$:

$$\psi(x_{k+1}) = \mathcal{K}[\psi(x_k); u_k; d_k]$$

Methodology – Basis Function

- One of the primary TISAC breakthroughs was applying Fourier encoding to allow the system to learn sudden, high-frequency changes without breaking the underlying physics.
- This same approach is adopted in my Comma AI solution, using a 1x7 Fourier basis function to lift the states:

$$z_k = \psi(x_k) = \begin{bmatrix} x_k \\ \sin(\pi x_k) \\ \cos(\pi x_k) \\ \sin(2\pi x_k) \\ \cos(2\pi x_k) \\ \sin(4\pi x_k) \\ \cos(4\pi x_k) \end{bmatrix}$$

Methodology - eDMD

- Next, approximate the Koopman Operator from open-loop baseline data via regularized least-squares regression (eDMD).

$$\Omega_k = \begin{bmatrix} z_k \\ u_k \\ d_k \end{bmatrix} = \begin{bmatrix} z_k \in \mathbb{R}^7 \\ u_k \in \mathbb{R}^1 \\ [v_{ego}, a_{ego}, bank]^T \in \mathbb{R}^3 \end{bmatrix}$$

- An augmented input-disturbance vector Ω_k can be defined in $\mathbb{R}^{11}$ as:

$$K = Z_{next} \Omega^T (\Omega \Omega^T + \lambda I)^{-1}$$

Where λ is a small regulation term

Methodology – State Space

The matrix K may be partitioned into $K = [A \mid B \mid G]$, creating the surrogate model:

$$z_{k+1} = Az_k + Bu_k + Gd_k$$

Where:

$A \in \mathbb{R}^{7 \times 7}$ is the state transition matrix

$B \in \mathbb{R}^{7 \times 1}$ is the control vector

$G \in \mathbb{R}^{7 \times 3}$ is the disturbance mapping

Methodology – Bayesian Methods

At this point we're all thinking – what in the world does this have to do with TISAC?

- To account for unmodeled, time-dependent parameters, the 7x11 elements of K are flattened into the vector $\theta \in \mathbb{R}^{77}$
- We create an **ensemble** of priors ($N = 50$), perturbed by Gaussian process noise Q :

$$\theta_k^{(i)} \sim N(\theta_{baseline}, Q), \text{ for } i \in \{1, 2, \dots, 50\}$$

- Each (of 50) ensemble members predicts a possible scalar lateral acceleration by projecting the measurement matrix $C = [1, 0, \dots, 0]$ onto the lifted prediction:

$$y_{pred,k}^{(i)} = C \left(K_{k|k-1}^{(i)} \Omega_k \right)$$

Methodology – Bayesian + MPC

- EnKF Update: The 77-parameter Kalman gain, $\mathcal{L}$ is computed from the scalar measurement innovation:

$$\mathcal{L} = P_{\theta,y}(P_{yy} + R)^{-1}$$

Where $P_{\theta,y} \in \mathbb{R}^{77}$ is the cross-covariance, isolating the gradient to shift all matrix parameters to match the physical space

- FINALLY: the calibrated ensemble of all 50 Koopman Matrices is passed to an MPC with a 5-step horizon. At each step k in horizon H , the optimizer rolls out 50 parallel futures via tensor multiplication:

$$Z_{k+1}^{(1:50)} = K_{\text{tensor}}^{(1:50)} \Omega_k^{(1:50)}$$

Methodology – Bayesian Methods

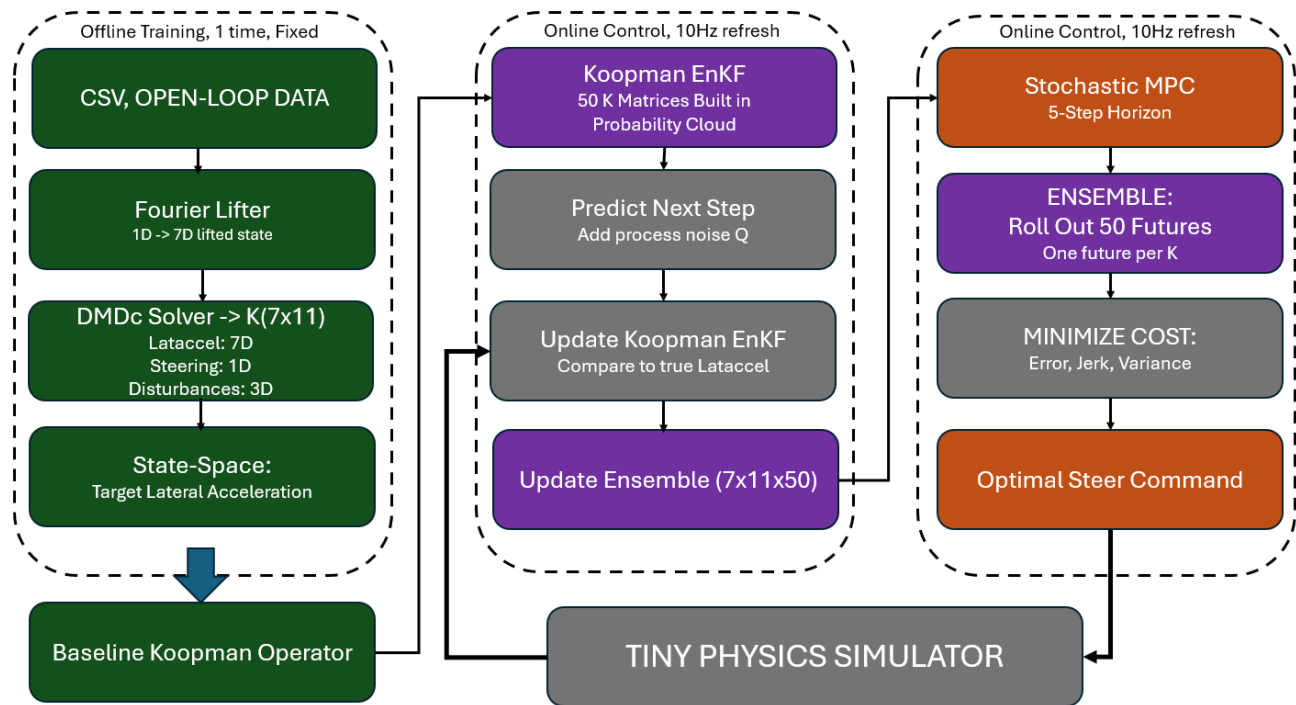
- At each step k , the MPC tests a candidate steering input, u
- All 50 K matrices will predict a unique lateral acceleration in response
- K varies every real-world time step (10Hz), but is constant within each horizon rollout
- If the 50 predictions cluster tightly, this implies:
 - Low variance $\rightarrow$ high system confidence $\rightarrow$ the optimizer ought to use that steering command
- If the 50 predictions spread widely this implies:
 - High variance $\rightarrow$ very low confidence $\rightarrow$ the uncertainty term creates a penalty such that the optimizer actively avoids it

Methodology – Bayesian Methods

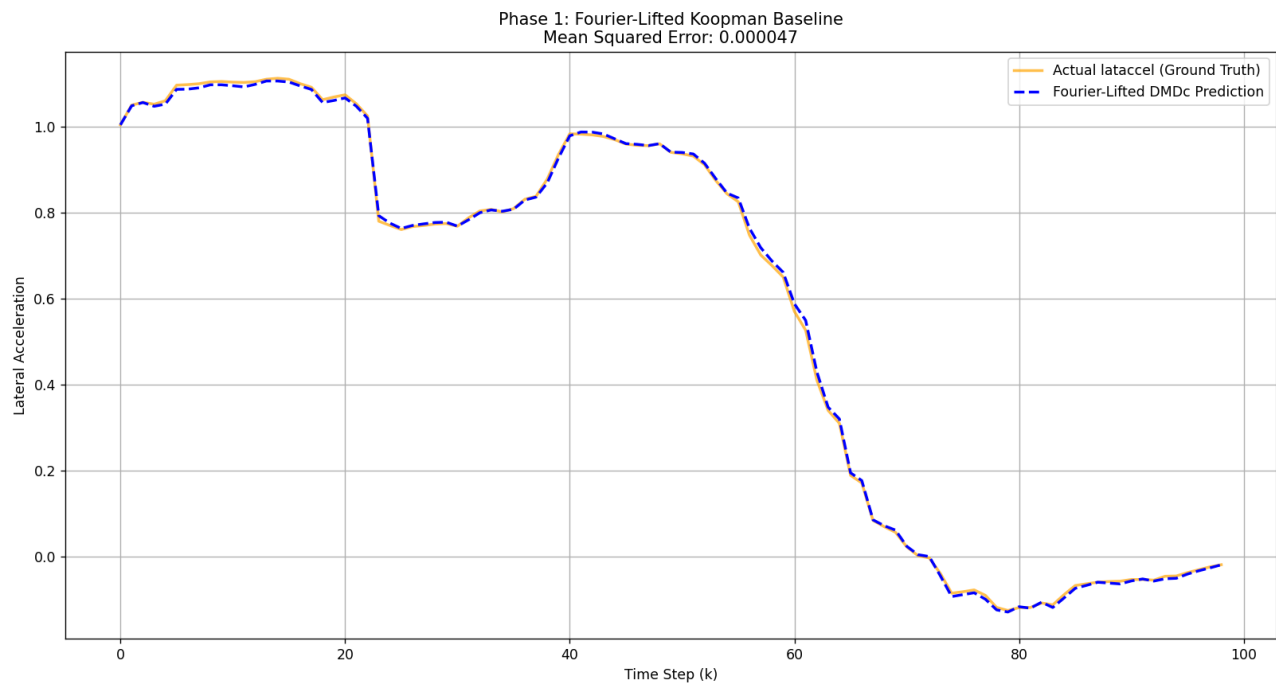
- Last but not least, an SLSQP solver is used to MINIMIZE the expected value of the tracking error while penalizing for epistemic uncertainty (variance) of the Koopman ensemble, pulled forward from our Bayesian methods:

$$J = \sum_{k=1}^H \left(w_{err}^2 (\mathbb{E}[y_{pred,k}] - y_{ref,k})^2 + w_{jerk} \Delta u_k^2 + w_{steer} u_k^2 + w_{uncert} \text{Var}(y_{pred,k}) \right)$$

Methodology Summary

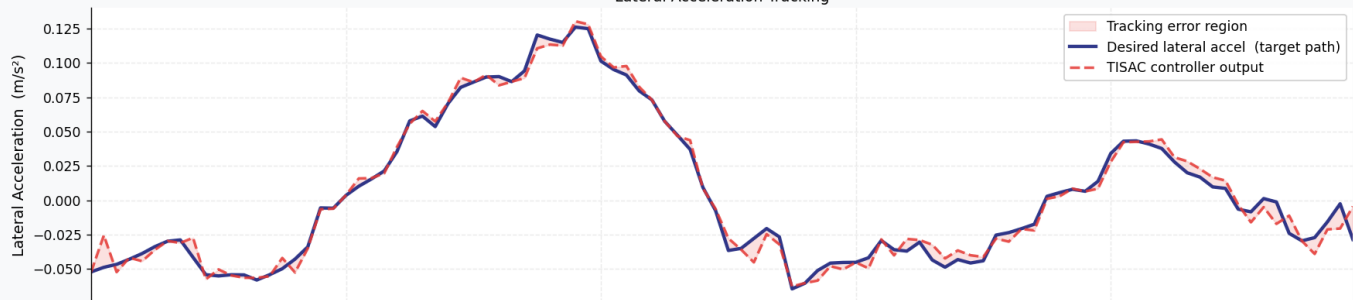


Results – Offline Training

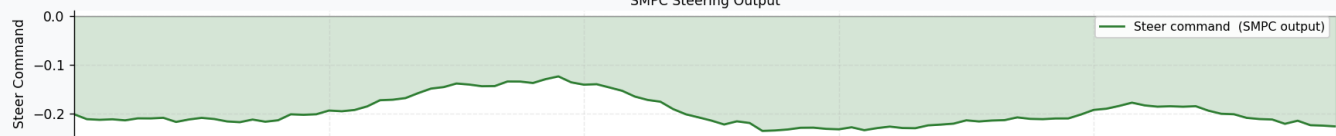


Results – Single Run

Lateral Acceleration Tracking



SMPC Steering Output



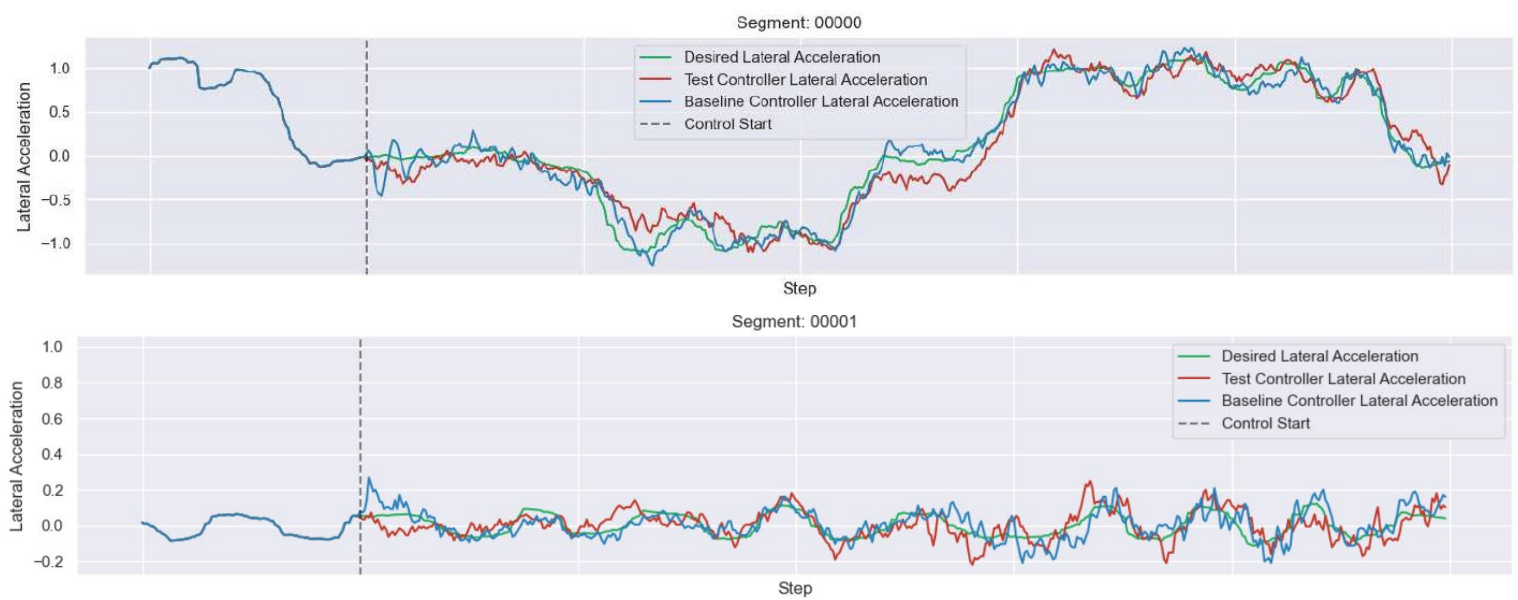
Results – Numerical

- Started with basic weights, untuned, large horizon
- Shorter horizon yields better results
- Optuna Study used to determine best penalty weights

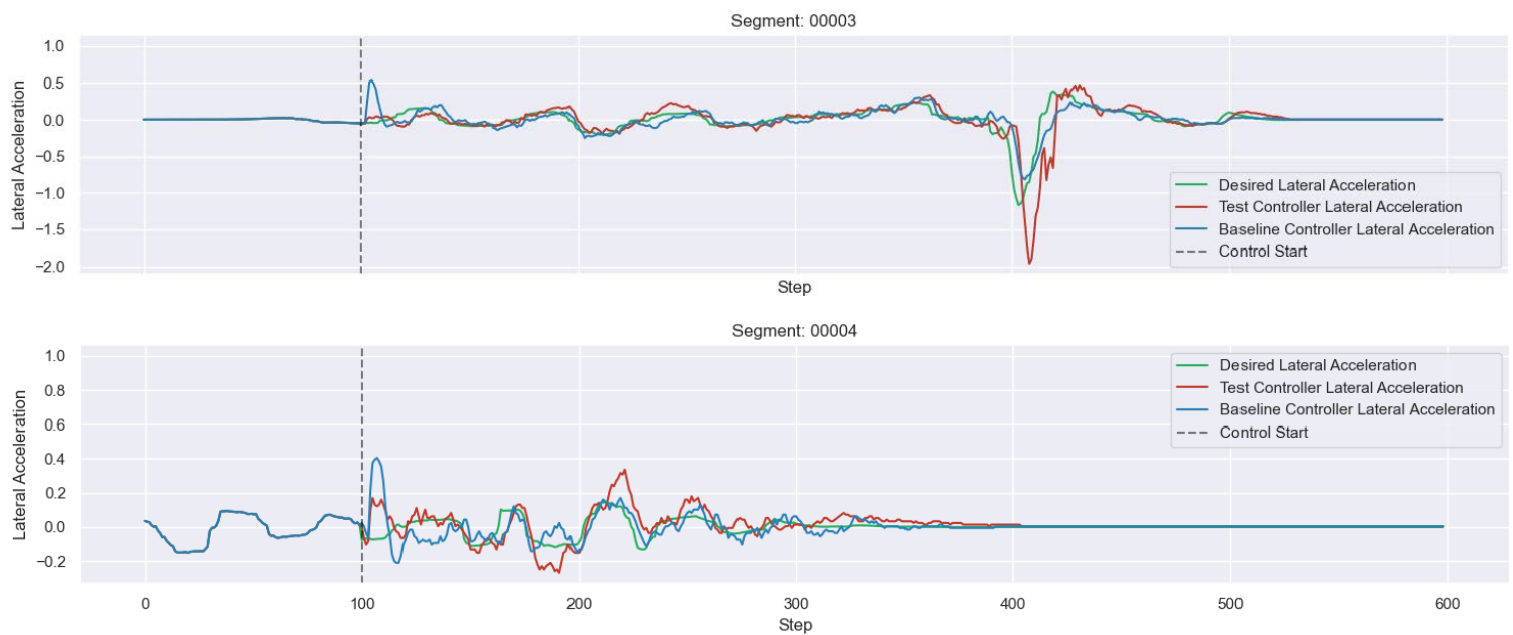
Method:	Lat_Accel Cost:	Jerk Cost:	Total Cost:	
PID BEST:	1.211	20.151	80.725	
INITIAL TISAC BEST:	3.065	15.470	168.722	
MID-POINT TISAC BEST:	2.900	16.375	156.15	
FINAL TISAC BEST:	1.193	32.498	92.137*	✓

*Score of 110 or lower required for leaderboard

Results – Graphical



Results – Graphical



Conclusion

- The basic TISAC formulation may be adopted to perform real-time, online control
- Parameter tuning is *extremely* important to obtaining reliable results
- A very strong, leaderboard-worthy result can be produced from this architecture, but requires thoughtful design and extensive testing
- It is highly complex and can be computationally expensive, so would likely require further studying and parallelization to be truly effective
- The controls-TISAC integration is successful in early trials, and has significant room for further development

References

- [1] Cho, K., Shao, K., & Ali, A. (2024). Run-indexed time-varying Bayesian optimization with positional encoding for auto-tuning of controllers: Application to a plasma-assisted deposition process with run-to-run drifts.
- [2] Doucet, A., de Freitas, N., & Gordon, N. (Eds.). (2001). Sequential Monte Carlo Methods in Practice. Springer New York.
- [3] Durbin, J., & Koopman, S. J. (2012). Time Series Analysis by State Space Methods (2nd ed.). Oxford University Press.
- [4] Higdon, D., Kennedy, M., Cavendish, J. C., Cafo, J. A., & Ryne, R. D. (2004). Combining field data and computer simulations for calibration and prediction. *SIAM Journal on Scientific Computing*, 26(2), 448–466.
- [5] Hoeting, J. A., Madigan, D., Raftery, A. E., & Volinsky, C. T. (1999). Bayesian model averaging: A tutorial. *Statistical Science*, 14(4), 382–417.
- [6] Juang, C. H., et al. (2013). Probabilistic model calibration for soil slope under rainfall: Effects of measurement duration and frequency in field monitoring. *Engineering Geology*.
- [7] Raftery, A. E., Gneiting, T., Balabdaoui, F., & Polakowski, M. (2005). Using Bayesian model averaging to calibrate forecast ensembles. *Monthly Weather Review*, 133(5), 1155–1174.
- [8] West, M., & Harrison, J. (1997). Bayesian Forecasting and Dynamic Models (2nd ed.). Springer-Verlag.